

Understanding LangChain Prompts: A Detailed Guide with Real-World Examples

When most beginners start learning LangChain, they often think a prompt is simply a message sent to ChatGPT. In reality, prompts are one of the most important components of any AI application.

The quality of an LLM's response depends heavily on how well you instruct it. Two developers can use the exact same model and build completely different-quality applications simply because one knows how to design prompts effectively while the other does not.

What is a Prompt?

A prompt is simply the instruction or input that you send to an LLM.

For example:

Write a short paragraph about Bangladesh.

This is a prompt.

Another example:

Explain Machine Learning in simple terms.

This is also a prompt.

An LLM cannot read your mind. It only understands what you communicate through the prompt.

The clearer your instructions are, the better the output will be.

Think of it like giving instructions to an employee.

If you say:

Create a report.

The task is vague.

But if you say:

Create a 300-word report about the impact of AI on education.

The task becomes much clearer.

The same principle applies when working with LLMs.

What is Temperature and Why Does It Matter?

Temperature is a parameter that controls how creative or predictable a model's responses will be.

A simple rule to remember:

Lower Temperature = More Predictable Output

Higher Temperature = More Creative Output

Temperature = 0

Suppose the prompt is:

What is the capital of France?

With Temperature set to 0, the model will almost always respond with:

Paris

Because it chooses the most likely answer every time.

Even if you ask the same question ten times, you'll receive nearly identical responses.

This behavior is called **Deterministic Output**.

Low temperature is commonly used for:

- Question Answering
- Data Analysis
- Information Extraction
- Summarization

In these tasks, accuracy is more important than creativity.

Temperature = 1.5 or 2

Now consider the prompt:

Write a short story about a dragon.

First response:

A dragon lived on a mountain and protected an ancient treasure.

Second response:

A young dragon dreamed of becoming the king of the skies.

Third response:

A lonely dragon traveled across the world searching for friends.

The prompt remains the same, but the outputs vary.

This happens because a higher temperature encourages the model to explore less likely word choices and generate more diverse responses.

High temperature is useful for:

- Story Writing
- Content Creation
- Marketing Copy
- Poetry
- Brainstorming

What is a Static Prompt?

A Static Prompt is hardcoded into your application.

Example:

prompt = "Write a summary about Artificial Intelligence."

This prompt never changes.

Regardless of what the user wants, the prompt remains fixed.

Imagine your website has 100 users.

One user wants a summary of Machine Learning.

Another wants a summary of Deep Learning.

A third wants a summary of NLP.

A static prompt cannot easily handle these different requests because it is fixed.

What is a Dynamic Prompt?

A Dynamic Prompt contains variables that can be replaced with user input.

Example:

template = "Write a summary about {topic}"

If the user provides:

Machine Learning

The final prompt becomes:

Write a summary about Machine Learning

If another user provides:

Deep Learning

The final prompt becomes:

Write a summary about Deep Learning

With a single template, you can generate thousands of different prompts.

This is why dynamic prompts are widely used in real-world AI applications.

Why Use PromptTemplate?

A common question is:

"Why use PromptTemplate when Python f-strings can already do this?"

For example:

```
topic = "Machine Learning"

prompt = f"Write a summary about {topic}"
```

This works perfectly fine.

However, PromptTemplate provides several advantages when building larger applications.

Validation

Consider this template:

```
template = """
Write a summary about {topic}
in {language}
"""
```

Suppose you only provide:

```
topic = "Machine Learning"
```

and forget to provide `language`.

PromptTemplate will immediately raise an error.

This makes debugging much easier.

Reusability

Templates can be created once and reused many times.

For example, if you're building a blog generator, the same template can generate thousands of blog posts.

LangChain Integration

PromptTemplate integrates directly with other LangChain components such as:

- Chains
- Agents
- Memory
- RAG Pipelines

Because of this, PromptTemplate is considered the standard approach in professional LangChain projects.

The Memory Problem in Chatbots

This is one of the most misunderstood topics among beginners.

Many people assume ChatGPT remembers everything.

In reality, LLMs do not have built-in memory.

Consider this conversation:

User:

My name is Rahim.

Assistant:

Nice to meet you, Rahim.

A few minutes later:

User:

What is my name?

If the previous messages are not included in the prompt, the model will have no idea who Rahim is.

The model only sees the current input.

It does not automatically remember previous conversations.

How Does Chatbot Memory Actually Work?

The trick is surprisingly simple.

Whenever a new question arrives, previous conversation history is included in the prompt.

Instead of sending only:

What is my name?

The application sends:

User: My name is Rahim.

Assistant: Nice to meet you, Rahim.

User: What is my name?

Now the model has enough context to answer:

Your name is Rahim.

This concept is known as **Chat History Management**.

The Three Types of Messages in LangChain

LangChain organizes conversations into three message types.

System Message

A System Message defines the AI's behavior, personality, and role.

Example:

You are a professional Python instructor.

Now the AI will try to answer as a Python teacher.

Another example:

You are a senior software architect.

Now the AI will focus on architecture-level explanations.

The System Message influences the entire conversation.

Human Message

A Human Message represents the user's input.

Example:

Explain decorators in Python.

This is a Human Message.

AI Message

An AI Message represents the model's response.

Example:

A decorator is a function that modifies the behavior of another function.

This is an AI Message.

Every real-world chatbot consists of these three message types.

What is ChatPromptTemplate?

When you're working with multiple messages instead of a single prompt, ChatPromptTemplate becomes useful.

Imagine building a chatbot that contains:

- A System Message
- Previous Conversation History
- The Current User Question

Managing all of these manually can become messy.

ChatPromptTemplate provides a structured way to organize them.

You can think of it as the chatbot version of PromptTemplate.

What is MessagesPlaceholder?

This is one of the most important concepts in LangChain.

Suppose your database contains the user's previous 50 messages.

You want to inject that conversation history into your prompt automatically.

This is where MessagesPlaceholder comes in.

Imagine your prompt structure looks like this:

System Message

Previous Chat History

Current User Question

The "Previous Chat History" section is replaced dynamically using MessagesPlaceholder.

LangChain fetches the history from storage and inserts it into the prompt.

As a result, the model receives all the necessary context.

A Real-World Example

Suppose the conversation history is:

User: My name is Rahim.

Assistant: Nice to meet you.

User: I live in Dhaka.

Assistant: That's great.

Now the user asks:

Where do I live?

When MessagesPlaceholder is used, the entire conversation history is included in the prompt.

The model can then answer:

You live in Dhaka.

Without the chat history, the model would not be able to answer correctly.